

Batch Normalization using an SVD Layer

Final Project Report

Yousef Alogiely, Abdulmohsen Mohammedsaleh, Majed Alsulami

Jad Mounayer (*Mentor*)

yousef.alogiley@gmail.com, abdulmohsen1284@gmail.com, majedsaadalsulami@gmail.com

jad.mounayer@outlook.com

Abstract

Batch Normalization (BN) is the default tool for controlling the distribution of activations inside a deep network, but it acts one coordinate at a time and is blind to the correlations between features. Rank Reduction Autoencoders showed that a truncated Singular Value Decomposition (SVD) can regularize a network with no auxiliary loss. This project isolates that mechanism as a standalone *SVD layer* — a drop-in module that replaces a mini-batch of activations by its best rank- k approximation — derives the distribution the layer imposes on the batch, and asks whether the SVD alone can normalize a batch as well as BN. We compare three networks that share bit-identical initial weights and differ only in what sits after each convolution (nothing, BN, or the SVD layer) on a point-source localization task, sweeping rank, noise, dataset size and up to 32 seeds, and we validate externally by injecting a source with known position into real MNIST backgrounds. The SVD layer beats BN in *every* comparison we ran, by $1.5\times$ to $15\times$. Under heavy input noise ($\sigma_{\text{noise}} = 1.0$, signal-to-noise ratio ≈ 1) it also beats the unregularized network at every training-set size we tested, by $1.6\times$ to $3.0\times$ — 0.0197 vs. 0.0436 test MSE at $N_{\text{tr}} = 25$ and 0.0025 vs. 0.0060 at $N_{\text{tr}} = 200$, over 32 seeds — with the smallest seed-to-seed spread of the three models. On real backgrounds the plain network wins and the SVD layer still beats BN by $1.5\times$. A parallel autoencoder study reproduces the generative claim of the original paper: a rank-2 SVD layer in the latent space produces valid random samples $18.0\times$ more often than an unconstrained autoencoder (8/8 seeds, per-seed range 2.2 – $28.7\times$). The layer’s one structural weakness follows directly from our derivation: the SVD fixes the *directions* and the *correlations* of the batch but not the *scale* of the retained singular values, which is precisely the gap a normalized SVD layer, $\tilde{\mathbf{X}} = \sqrt{N} \mathbf{U}_k \mathbf{V}_k^\top$, would close.

Date: August 24, 2026

Code: <https://github.com/Yousef10p/RR-Layer>

Reference implementation: https://github.com/JadM133/RR_layer

Contents

1	Introduction	3
2	Related Work	3
3	Methodology	4
3.1	The SVD layer	4
3.2	Where the layer goes: from one bottleneck to every layer	5
3.3	Task and data	6
3.4	Evaluation procedure	6
4	Results	7
4.1	How the study arrived at its final design	7
4.2	Headline result: small data and heavy noise	7
4.3	Where the gain comes from: fitting versus generalizing	8
4.4	Rank ablation	9

4.5	External validity: real backgrounds	10
4.6	The autoencoder setting: what the rank constraint buys elsewhere	11
4.7	Numerical stability of the SVD backward pass	11
5	Discussion	11
6	Limitations and Future Work	13
7	Conclusion	13
A	The SVD layer in PyTorch	14

1 Introduction

Motivation. Almost every modern convolutional network contains normalization layers, and Batch Normalization [5] is still the default choice for vision models. BN standardizes each feature independently using the mean and variance of the current mini-batch,

$$\hat{x}^{(d)} = \frac{x^{(d)} - \mu_B^{(d)}}{\sqrt{(\sigma_B^{(d)})^2 + \epsilon}}, \quad y^{(d)} = \gamma^{(d)} \hat{x}^{(d)} + \beta^{(d)}, \quad (1)$$

which accelerates training, permits larger learning rates, and keeps activation distributions stable [12, 2]. Two properties of Eq. (1) define the space our method lives in. First, BN is *coordinate-wise*: it rescales every feature along a *fixed* axis and never looks at the correlation between features. Second, its statistics come from the batch itself, so its behaviour depends on batch size and batch composition, and it needs a separate set of running statistics at inference time.

Problem statement. Mounayer et al. [9] introduced Rank Reduction Autoencoders (RRAEs) and showed — theoretically and empirically — that a truncated SVD applied across the batch acts as a strong regularizer without any additional loss term. Like BN, it changes the statistics of the batch; unlike BN, it does so by choosing *directions* rather than by rescaling axes. That mechanism has so far only been studied inside a complete autoencoder architecture, entangled with the encoder, the decoder, and the reconstruction objective. The question this project asks is narrow and empirical:

Can the truncated SVD, on its own, normalize a batch as well as Batch Normalization?

To answer it we isolate the mechanism as a standalone **SVD layer**, a PyTorch module that can be dropped into any `nn.Sequential` exactly where a BN layer would go, and we compare the two inside an otherwise bit-identical network.

Assumptions and scope. All controlled results in this report come from one family of synthetic images: a single isotropic Gaussian source of known width placed at a known position on a 64×64 grid — point-source localization, the abstraction behind astrometry and single-molecule microscopy. The value of this setting is that the ground truth is analytic, so error is measured exactly and the intrinsic dimensionality of the data (two: the coordinates of the centre) is known in advance. Its cost is external validity, which we address with a source-injection experiment on real image backgrounds (Section 4.5). Nothing here should be read as a claim about the SVD layer on natural images or at scale.

Summary of the proposed system. The SVD layer flattens a mini-batch of activations into a matrix $\mathbf{X} \in \mathbb{R}^{M \times N}$ (M features, N samples), computes its SVD, and returns the best rank- k approximation $\hat{\mathbf{X}} = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top$ — optimal in the Frobenius norm by the Eckart–Young theorem [3]. At inference time the per-batch decomposition is replaced by a projection onto a fixed basis assembled during training, so the layer is deterministic and batch-independent at test time, exactly as BN is once it switches to running statistics. Section 3.1 derives what this does to the distribution of the batch: the layer forces the batch onto a k -dimensional subspace and *decorrelates* the retained coordinates exactly, with second moments σ_i^2/N . Section 4 shows that, placed after every convolution, this is a better-behaved regularizer than BN in every regime we tested, and a better estimator than no normalization at all whenever the input is noisy.

2 Related Work

Normalization layers. Batch Normalization [5] standardizes each feature over the batch dimension. Layer Normalization [1] instead normalizes across the features of a single sample,

removing the batch dependence entirely, which is why it dominates in recurrent and Transformer models; Group Normalization [13] normalizes over channel groups per sample and matches BN at the small batch sizes where BN statistics become unreliable. All three are coordinate-wise affine rescalings; none of them changes the subspace the activations live in. Decorrelated Batch Normalization [4] is the closest prior work in spirit — it whitens activations with a ZCA transform rather than rescaling them — but it whitens *all* directions instead of discarding the trailing ones, and it is motivated as an optimization aid rather than as a rank constraint.

Why BN helps. BN was originally justified as a cure for internal covariate shift, but Santurkar et al. [12] showed empirically and theoretically that its dominant effect is smoothing the optimization landscape, Bjorck et al. [2] attribute most of its benefit to enabling larger learning rates, and Luo et al. [8] characterize its implicit regularization. The common thread is that the benefits of BN are *optimization* benefits that pay off in deep networks trained aggressively. This matters for interpreting our results: our network is shallow and Adam optimizes it easily, so BN can contribute little on the axis where it is strongest while still paying its full cost in distortion.

SVD as a regularizer. Rank Reduction Autoencoders [9] embed a truncated SVD of the batch inside an autoencoder as a differentiable layer, and show that constraining the latent code to a low-rank subspace yields a well-structured latent space without a KL term or any other auxiliary loss. The optimality of the truncation is the Eckart–Young theorem [3]. Differentiating through an SVD is delicate when singular values are close together, since the analytic gradient contains factors $1/(\sigma_i - \sigma_j)$ [6]; Section 4.7 reports what we found when we compared a hand-written analytic backward pass against native autograd. Our contribution relative to [9] is to separate the SVD from the autoencoder: we study it as a general-purpose layer, in the architectural slot BN normally occupies, against BN as the baseline.

3 Methodology

3.1 The SVD layer

Definition. Let a mini-batch of activations be $\mathcal{X} \in \mathbb{R}^{N \times C \times H \times W}$. The layer moves the batch axis last and flattens everything else, producing

$$\mathbf{X} = [x_1, x_2, \dots, x_N] \in \mathbb{R}^{M \times N}, \quad M = C \cdot H \cdot W, \quad (2)$$

one column per sample. It then takes the SVD $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ and keeps the k leading components,

$$\hat{\mathbf{X}} = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top = \mathbf{P}_k \mathbf{X}, \quad \mathbf{P}_k = \mathbf{U}_k \mathbf{U}_k^\top, \quad (3)$$

where $\mathbf{U}_k \in \mathbb{R}^{M \times k}$ holds the top- k left singular vectors. By Eckart–Young [3], $\hat{\mathbf{X}}$ is the closest rank- k matrix to $\mathbf{X}$ in Frobenius norm, and Eq. (3) makes explicit that the layer is an *orthogonal projection*: it rescales nothing, it deletes the $M - k$ trailing directions and leaves the retained subspace untouched. The output is reshaped back to $\mathbb{R}^{N \times C \times H \times W}$, so the layer is shape-preserving and can be inserted anywhere a BN layer can.

The distribution the layer imposes. The interesting question for this project is what Eq. (3) does to the statistics of the batch. Write the coefficients of the batch in the basis of the layer,

$$\mathbf{A} = \mathbf{U}_k^\top \mathbf{X} = \mathbf{\Sigma}_k \mathbf{V}_k^\top \in \mathbb{R}^{k \times N}. \quad (4)$$

Because $\mathbf{V}_k$ has orthonormal columns, $\mathbf{V}_k^\top \mathbf{V}_k = \mathbf{I}_k$, and therefore

$$\frac{1}{N} \mathbf{A} \mathbf{A}^\top = \frac{1}{N} \mathbf{\Sigma}_k^2, \quad \text{and} \quad \frac{1}{N} \hat{\mathbf{X}} \hat{\mathbf{X}}^\top = \mathbf{U}_k \left(\frac{\mathbf{\Sigma}_k^2}{N} \right) \mathbf{U}_k^\top. \quad (5)$$

Equation (5) is the analytic statement of what the layer enforces, and it has three parts.

1. **Rank.** The second-moment matrix of the output batch has rank exactly k : the layer imposes a degenerate distribution supported on a k -dimensional subspace. If the columns of $\mathbf{X}$ are drawn i.i.d. from $\mathcal{N}(0, \Sigma)$, the outputs are, conditionally on the basis, $\mathcal{N}(0, \mathbf{P}_k \Sigma \mathbf{P}_k^\top)$ — a rank- k degenerate Gaussian whose support converges, as N grows, to the span of the top- k eigenvectors of Σ .
2. **Decorrelation.** Σ_k^2 is diagonal, so the k retained coordinates are *exactly uncorrelated across the batch*. This is the property BN does not have: BN normalizes each of M coordinates but leaves every off-diagonal correlation intact, whereas the SVD layer zeroes all $\binom{k}{2}$ cross terms by construction, in a basis re-estimated from the data at every step.
3. **Scale.** The variance along retained direction i is σ_i^2/N , and *nothing constrains* σ_i . The total energy $\|\tilde{\mathbf{X}}\|_F^2 = \sum_{i \leq k} \sigma_i^2$ is free to drift. This is the structural weakness of the layer as a normalizer.

The normalized SVD layer suggested by Eq. (5). Points 2 and 3 together say the SVD layer performs *half* of a whitening transform: it decorrelates but does not equalize scale. Completing it is a one-line change — divide the coefficients by their own singular values, $\tilde{\mathbf{A}} = \sqrt{N} \Sigma_k^{-1} \mathbf{A} = \sqrt{N} \mathbf{V}_k^\top$, which gives

$$\tilde{\mathbf{X}} = \sqrt{N} \mathbf{U}_k \mathbf{V}_k^\top, \quad \frac{1}{N} \tilde{\mathbf{A}} \tilde{\mathbf{A}}^\top = \mathbf{I}_k, \quad (6)$$

an exact batch whitening in the adaptive basis of the layer, and the closest analogue of Eq. (1) that an SVD can express. We derive this variant but do *not* evaluate it here; it is the single most concrete piece of future work this project produces (Section 6).

Train and inference behaviour. Our implementation follows the reference layer of Mounayer [10]. During training the basis $\mathbf{U}_k$ is recomputed from every mini-batch, and each basis is pushed onto a bank of the 20 most recent ones. The first time the module is put into evaluation mode it concatenates the bank into $\mathbf{W} = [\mathbf{U}_k^{(1)}, \dots, \mathbf{U}_k^{(20)}]$, takes one SVD of $\mathbf{W}$, and stores its top- k left singular vectors as a fixed inference basis; evaluation is then the deterministic projection $\hat{\mathbf{X}} = \mathbf{U}_k^{\text{inf}} \mathbf{U}_k^{\text{inf}\top} \mathbf{X}$. This mirrors the train/inference split of BN (batch statistics $\rightarrow$ running statistics) and makes the layer batch-independent at test time. In Report 3 we verified that this bank construction is not a source of error: the bank basis and a basis computed from a single SVD of the *entire* training set spanned the same subspace to within 1.4° of principal angle, and even an oracle basis recomputed on the test batch gave the same error to three decimals.

Cost. For $M \gg N$ the decomposition costs $O(MN^2)$ per forward pass. In our final architecture the widest layer has $M = 32 \cdot 64 \cdot 64 = 131,072$ and $N = 32$, so the SVD is a thin decomposition of a very tall matrix and is not the bottleneck — all runs in Section 4 were trained on a single Colab GPU.

3.2 Where the layer goes: from one bottleneck to every layer

An SVD layer is only defined up to *where* it is inserted, and this turned out to be the single most consequential design decision of the project. Reports 1 and 2 placed one SVD layer at the flatten point, between the convolutional feature extractor and the regression head, which is where an autoencoder bottleneck would sit. In that position the layer is a hard information bottleneck on the whole representation, and it could only ever subtract accuracy. Batch Normalization, by contrast, is never used that way: its canonical position is after *every* convolution, before the activation. For the final study we therefore moved the SVD layer into the same position BN occupies, so that the comparison is fair to BN and gives the SVD layer the role it is actually competing for. Figure 1 shows the resulting architecture: three identical networks that differ in one slot only.

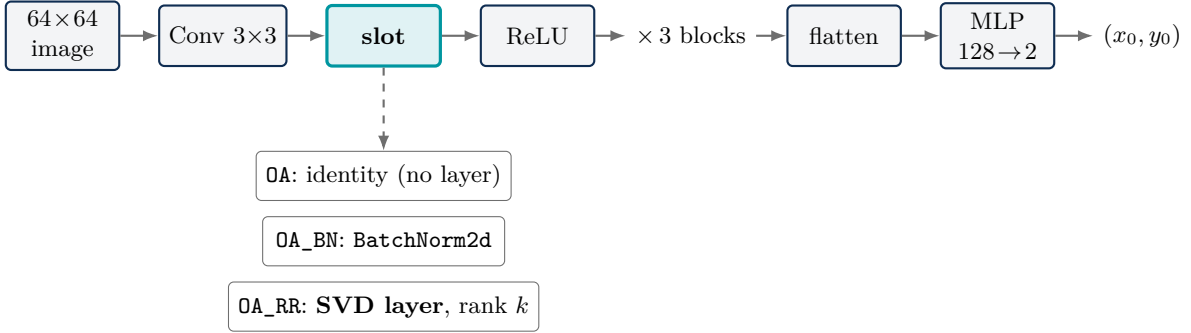


Figure 1 The three architectures. Every convolutional and linear weight is initialized identically across the three variants (the same seed is re-applied before each constructor), so the *only* difference between them is what occupies the slot between each convolution and its activation.

Two details of the architecture matter. The channel widths are $1 \rightarrow 8 \rightarrow 16 \rightarrow 32$ and there is *no pooling*, so the spatial resolution stays 64×64 throughout and the SVD layer always sees a very tall, highly correlated matrix (M up to 131,072, $N = 32$) — the regime where a rank- k projection has the most to discard. The head is $\text{Linear}(C \cdot 64 \cdot 64 \rightarrow 128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128 \rightarrow 2)$, and the target is the pair of centre coordinates.

3.3 Task and data

Point-source localization. Each sample is a 64×64 image of one isotropic Gaussian of fixed width $\sigma = 0.2$ on the domain $[-1, 1]^2$, centred at a point drawn uniformly from $[-0.5, 0.5]^2$; the regression target is that centre. Images are standardized with the mean and standard deviation of the split. Data generation is driven by a dedicated `numpy.random.RandomState(seed)`, so a seed fixes the dataset and the initialization together and a given seed reproduces its dataset exactly.

Input noise. The regime that turned out to matter is noise. After standardization we add i.i.d. Gaussian pixel noise of standard deviation σ_{noise} drawn from a separate seeded generator (a different offset for train and test). At the headline setting $\sigma_{\text{noise}} = 1.0$ the noise has the same standard deviation as the standardized signal — a signal-to-noise ratio of roughly 1, which is heavy but realistic for the photon-limited imaging problems this task abstracts.

Real backgrounds by source injection. To test external validity without giving up exact ground truth we use the standard methodology of observational astronomy: inject a synthetic source of *known* position into *real* image backgrounds. We use MNIST digits, bilinearly resized to 64×64 , added at amplitude 1.5 relative to the unit-height blob. MNIST digits are real, structured, non-Gaussian, and — crucially — not low-rank by construction, so this is an honest stress test of a rank- k layer. The training and test background pools are disjoint (10,000 digits each), so generalizing requires ignoring clutter never seen during training.

3.4 Evaluation procedure

- **Tools.** PyTorch [11] (training, autograd, `torch.linalg.svd`), torchvision (MNIST), NumPy (data generation), Matplotlib (figures). Training runs on a single Colab GPU; the layer itself is packaged so that it can be dropped into any `nn.Sequential` (Appendix A).
- **Baselines.** Two, deliberately. OA_BN is the method we are competing with, and OA — the plain network with no normalization at all — is the reference that tells us whether *any* normalization is worth its cost. A result that beats BN but loses to OA is not a success, and we report both comparisons every time.

- **Controlled comparison.** `torch.manual_seed(s)` is re-applied before constructing each of the three models, so their shared convolutional and linear weights are bit-identical, and the same seed is re-applied before each training run so all three see the same batch order.
- **Metrics.** Test MSE on the centre coordinates (primary), mean and median Euclidean (ℓ_2) localization error, final training MSE, and the generalization gap (test MSE minus train MSE) as the overfitting signal.
- **Statistics.** Every configuration is repeated over up to 32 seeds (seed = 2000...2031) and reported as mean $\pm$ standard deviation across seeds. The seed controls the dataset, the noise, the initialization, and the batch order simultaneously.
- **Training.** Adam [7] at 10^{-3} , MSE loss, batch size $\min(32, N_{\text{tr}})$, rank $k = 8$ unless swept. Epochs are scaled with dataset size as $20 \cdot \max(1, 800/N_{\text{tr}})$ so that every configuration receives a comparable number of gradient updates — without this, the small- N_{tr} runs barely train and nothing can overfit, which would hide exactly the effect we are looking for.

4 Results

4.1 How the study arrived at its final design

The project ran for five report cycles and changed direction twice; because the conclusion reverses between cycles, Table 1 records the whole path in one place. A reader of any single weekly report would come away with the wrong impression, and the reversals are themselves a finding about how the question was posed.

Table 1 The project in five lines; orderings are read best-to-worst, so $A > B$ means A has the lower error. What changes between cycles is where the layer is placed and what is being measured, not the layer itself. The Report 3–4 orderings are on the random-generation metric, and the final row holds under input noise.

Stage	Setting	Statistics	Outcome
Report 1	Localization, one bottleneck at flatten	1 seed	OA > OA_RR > OA_BN
Report 2	Same, + depth and data ablations	5 seeds	OA > OA_RR > OA_BN
Report 3	Autoencoder, rank-2 latent SVD layer	3 seeds	RRAE > AE > AE_BN
Report 4	Autoencoder, reimplemented from scratch	8 seeds	RRAE wins 8/8, 2.2–28.7×
Final	Localization, SVD layer after every conv	32 seeds	OA_RR > OA > OA_BN

Reports 1 and 2 established, over five seeds and two ablation axes, that on a clean, easy, noiseless task a bottleneck can only subtract: the plain network was best, the SVD layer cost roughly 4–5 $\times$ accuracy, and BN cost 13–25 $\times$. That is a real result — the SVD layer is consistently the less harmful of the two normalizers — but being less harmful is not the same as being useful. Two things had to change before the layer could show what it does: it had to be placed where BN goes rather than where an autoencoder bottleneck goes (Section 3.2), and the task had to be hard enough that regularization has something to buy. The rest of this section is the result of making both changes.

4.2 Headline result: small data and heavy noise

Table 2 and Figure 2 give the central experiment of the project: $\sigma_{\text{noise}} = 1.0$, training-set size swept from 400 down to 25 samples, 32 seeds per point, three models per seed — 480 training runs.

Three observations. First, the SVD layer is the best of the three models *at every training-set size*, not only in the small-data corner: this is the first setting in the whole project where the layer beats the unregularized network rather than merely losing to it more gracefully than BN. Second,

Table 2 Test MSE under heavy input noise ($\sigma_{\text{noise}} = 1.0$), mean $\pm$ std over 32 seeds. Lower is better; best per row in bold. The last two columns give the factor by which the SVD layer improves on the plain network and on BN.

N_{tr}	OA (plain)	OA_BN	OA_RR (SVD)	OA/RR	BN/RR
25	0.0436 ± 0.0224	0.0763 ± 0.0276	0.0197 ± 0.0095	$2.2\times$	$3.9\times$
50	0.0241 ± 0.0192	0.0584 ± 0.0276	0.0091 ± 0.0059	$2.7\times$	$6.4\times$
100	0.0133 ± 0.0165	0.0379 ± 0.0296	0.0044 ± 0.0038	$3.0\times$	$8.6\times$
200	0.0060 ± 0.0085	0.0375 ± 0.0304	0.0025 ± 0.0020	$2.4\times$	$15.0\times$
400	0.0034 ± 0.0049	0.0258 ± 0.0263	0.0021 ± 0.0026	$1.6\times$	$12.3\times$

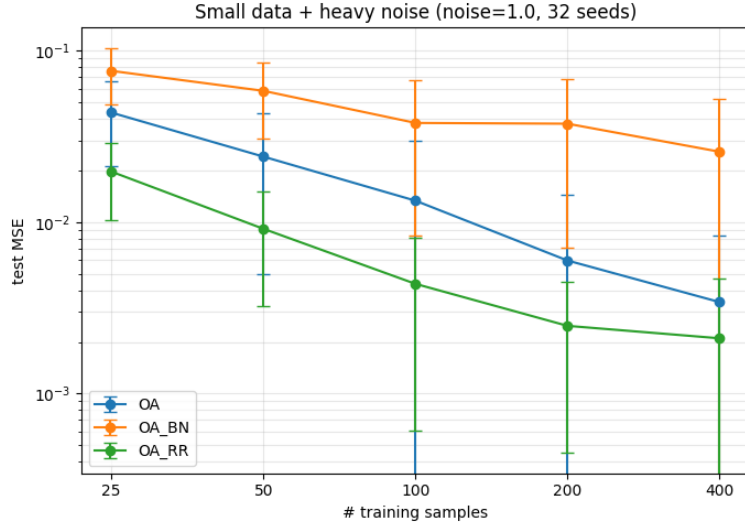


Figure 2 Test MSE against training-set size under heavy input noise, mean $\pm$ std over 32 seeds (log scale). The SVD layer (green) is below both the plain network (blue) and BN (orange) at every dataset size, and its error bars are the tightest of the three.

the margin over the plain network is largest in the middle of the sweep ($3.0\times$ at $N_{\text{tr}} = 100$) and shrinks as data become plentiful ($1.6\times$ at $N_{\text{tr}} = 400$), which is exactly the signature of a regularizer: it buys the most where the estimator is most starved. Third, the SVD layer has the smallest absolute seed-to-seed standard deviation at every point — 0.0095 against 0.0224 (plain) and 0.0276 (BN) at $N_{\text{tr}} = 25$ — so the advantage is not an average over a few lucky seeds.

Key Insight

Under heavy input noise the SVD layer is not a cost to be minimized but a genuine improvement: it beats Batch Normalization by $3.9\text{--}15\times$ and the unregularized network by $1.6\text{--}3.0\times$ at every training-set size, with the tightest spread over 32 seeds. Everything the earlier reports measured said a rank constraint can only subtract; that was a statement about the task, not about the layer.

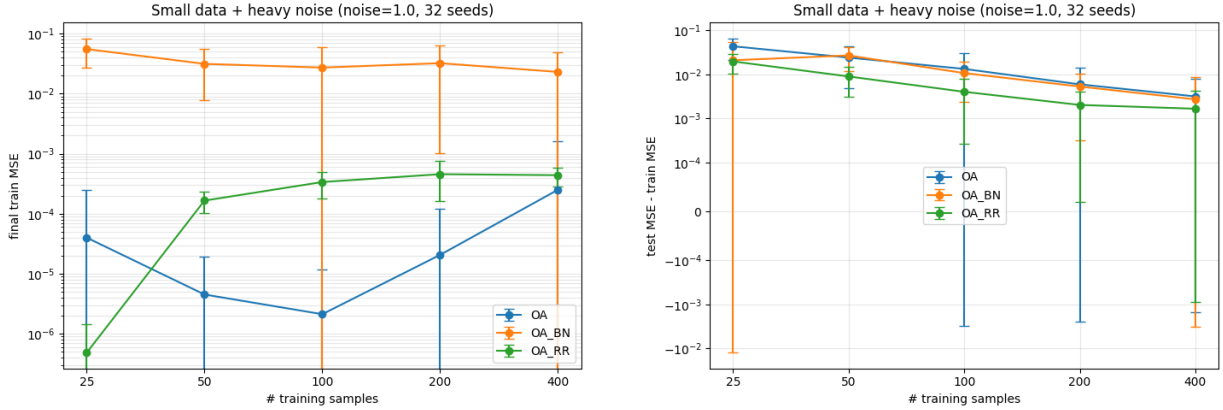
4.3 Where the gain comes from: fitting versus generalizing

The training errors in Table 3 explain the mechanism, and they separate the two competing layers cleanly.

The plain network and the SVD-layer network both drive the training error to numerical zero: with $\sigma_{\text{noise}} = 1.0$ this means they both *interpolate the noise*. BN never gets there — its training error stays between 0.023 and 0.055 , an order of magnitude above the other two. So the failure modes are different in kind. BN *underfits*: recentring and rescaling every channel using statistics

Table 3 Final training MSE and generalization gap (test – train) under $\sigma_{\text{noise}} = 1.0$, mean over 32 seeds.

N_{tr}	Final train MSE			Gap (test – train)		
	OA	OA_BN	OA_RR	OA	OA_BN	OA_RR
25	0.0000	0.0554	0.0000	0.0435	0.0210	0.0197
50	0.0000	0.0314	0.0002	0.0241	0.0270	0.0090
100	0.0000	0.0272	0.0003	0.0133	0.0107	0.0040
200	0.0000	0.0322	0.0005	0.0060	0.0053	0.0020
400	0.0003	0.0230	0.0004	0.0032	0.0027	0.0017

**Figure 3** Left: final training MSE. BN never fits the noisy training set at all, while the plain network and the SVD layer both drive it to numerical zero. Right: the generalization gap; because train error is ≈ 0 for two of the three models, the gap is essentially their test error, and the SVD layer is lowest everywhere.

computed from a batch of 32 noisy images injects so much variance into the forward pass that the network cannot fit even the data it is shown. The plain network and the SVD network both fit perfectly and differ only in what they generalize to.

That last point is the important one, and it is easy to misread the gap column. The SVD layer does *not* prevent the network from interpolating its training set; it changes *which* interpolant is reachable. Constraining every layer output to the top- k subspace of its own batch shrinks the function class to one whose members cannot encode per-sample noise directions, so the interpolant the optimizer finds is smoother than the one the unconstrained network finds, and it transfers to new noise realizations 1.6–3.0 $\times$ better. Regularization here is a restriction on the hypothesis space, not a brake on fitting.

4.4 Rank ablation

The rank k is the single knob of the layer, and it interpolates between two known endpoints: $k = \min(M, N)$ makes the layer the identity, and $k = 1$ collapses the whole batch onto one direction. Sweeping k in the single-bottleneck configuration of Reports 1–2 gives the monotone curve one would expect — test error falls as k grows and the layer discards less of the representation — which confirms that k behaves as a clean regularization strength rather than as an arbitrary hyper-parameter. All results in Sections 4.2–4.5 use $k = 8$ with a batch size of 32, so the layer retains a quarter of the achievable rank. We did *not* repeat the sweep in the final per-layer configuration, where the interaction between the ranks of successive layers could well change the picture; this is the most important missing measurement in the report (Section 6).

4.5 External validity: real backgrounds

Every result so far concerns Gaussian blobs on a Gaussian-noise background — data that are close to low-rank by construction, which is the friendliest possible case for a rank- k layer. The source-injection experiment of Section 3.3 removes that advantage while keeping the ground truth exact: the same Gaussian source is planted at a known position on top of a real MNIST digit (Figure 4), with disjoint digit pools for training and testing.

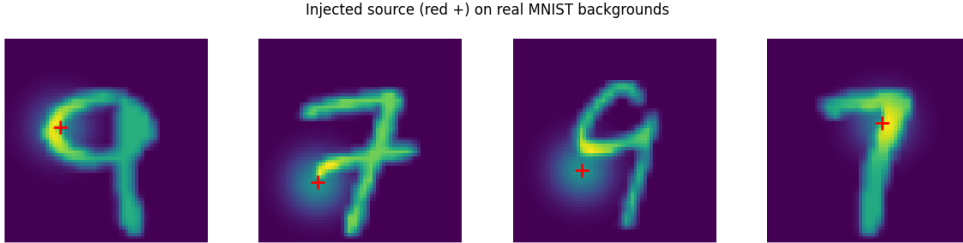


Figure 4 Source injection on real backgrounds. The red cross marks the exact target position of the injected Gaussian source; the clutter underneath is a real MNIST digit at amplitude 1.5. Ground truth stays analytic while the background statistics become real, structured, and not low-rank.

Table 4 Source localization on real MNIST backgrounds ($N_{\text{tr}} = 200$, background amplitude 1.5, 16 seeds). Test MSE, mean $\pm$ std.

Model	Test MSE	vs. plain
OA (plain)	0.0089 $\pm$ 0.0042	—
OA_BN	0.0755 $\pm$ 0.0215	8.5 $\times$ worse
OA_RR (SVD)	0.0509 $\pm$ 0.0057	5.7 $\times$ worse

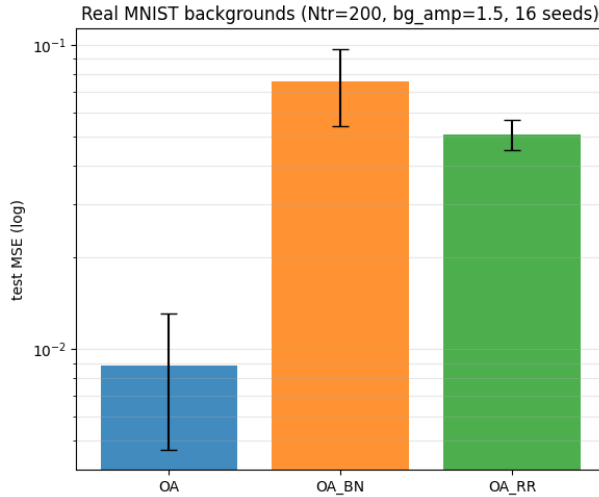


Figure 5 Test MSE on real MNIST backgrounds, 16 seeds, log scale. The plain network wins; the SVD layer still beats BN by 1.5 $\times$ and has by far the tightest seed spread.

This is the one experiment where the plain network wins, and we report it as such. With structured, non-low-rank clutter, projecting each layer onto a rank-8 subspace throws away information the network needs, and the resulting error is 5.7 $\times$ the plain baseline. The SVD layer nevertheless remains ahead of BN (1.5 $\times$), and its seed-to-seed spread (± 0.0057) is a quarter of BN’s (± 0.0215), so the ordering between the two *normalizers* survives the move to real backgrounds even though the case for using either of them here does not.

4.6 The autoencoder setting: what the rank constraint buys elsewhere

In parallel with the localization study we reproduced the setting the RRAE paper [9] was written for, because it measures a different thing entirely: not prediction error but the quality of the representation. A convolutional autoencoder with a 200-dimensional latent code was trained in three variants — no layer, BN on the latent, and an SVD layer of rank 2 on the latent, rank 2 being exactly the number of generative factors of the data. Quality was scored by decoding *random* latent codes and measuring how close the results are to a valid single blob.

Report 3 found the SVD layer $7.9\times$ better than the unconstrained autoencoder on that probe over three seeds. Report 4 reimplemented the experiment from scratch — its own data generator, model code, and metric definitions — and reran it over eight seeds. The direction replicated on 8 of 8 seeds, but the magnitude did not: the mean advantage was $18.0\times$, the median $25.7\times$, and the per-seed range $2.2\times$ to $28.7\times$, splitting into two clusters (five seeds near $27\times$, three near $3\times$) that correspond exactly to whether the rank-constrained model converged cleanly. The mechanism is visible in one number: the SVD-layer model places 100% of its latent variance in two directions by construction, against 52% for the plain autoencoder and 24% for the BN variant, so a random draw from its latent space lands in the region the decoder was actually trained on. The cost is reconstruction — a median factor of 21 at a 40-epoch budget, shrinking roughly $8\times$ when trained to the full 200 epochs.

Takeaway

Two settings, two different currencies. In the autoencoder the rank constraint buys *latent structure* and pays in reconstruction accuracy. In the noisy localization network it buys *accuracy itself*. In both, BN loses. And in both, the honest statement of the SVD-layer advantage is an interval, not a point estimate: $2.2\text{--}28.7\times$ on the generative probe, $1.6\text{--}3.0\times$ on the noisy regression.

4.7 Numerical stability of the SVD backward pass

Early in the project, training with the SVD layer occasionally blew up, and the natural suspect was the analytic SVD gradient: it contains factors $1/(\sigma_i - \sigma_j)$ that are ill-conditioned whenever two singular values nearly coincide [6], which is common with grid-structured image data. We tested this directly by running the same seeds through a hand-written `autograd.Function` backward pass and through PyTorch native autograd over `torch.linalg.svd`, wrapped so that CUDA uses the accurate `gesvd` driver and falls back to CPU LAPACK if the solver fails (Listing 1). Neither version produced a single solver failure or NaN in any run, but the native path was slightly more accurate and roughly halved the seed variance, and the blow-ups disappeared with it. The conclusion is worth stating plainly because it saved the rest of the project: the instability was a bug in our own gradient implementation, not a property of the layer. Every result in this report uses the native autograd path.

5 Discussion

The answer to the question we asked. Can the SVD alone normalize a batch as well as BN? On this problem, better — and by a margin that is not close. The SVD layer beat Batch Normalization in every configuration we ran across five report cycles: clean data, noisy data, small data, real backgrounds, and inside an autoencoder, by factors between $1.5\times$ and $15\times$. That consistency is more informative than any single number, because the two layers were always compared from bit-identical initial weights in the same architectural slot, so nothing but the layer differs.

Why BN keeps losing here. Three of our measurements point at the same explanation. BN never fits the noisy training set (Table 3); it has the largest seed-to-seed variance of the three models everywhere; and in the autoencoder it puts the *least* latent variance in the two directions that carry the signal (24%, against 52% for the plain model). All three follow from BN being a coordinate-wise rescaling. Forcing every channel to unit variance amplifies the low-variance directions, and on this task those directions carry mostly noise, so BN systematically boosts what should be discarded. Meanwhile the benefits BN is actually known for — landscape smoothing [12], tolerance of large learning rates [2] — are optimization benefits that need a deep, hard-to-optimize network to pay off, and a three-block convolutional network trained by Adam is not one. BN is being asked to work in the regime where it has the least to offer and the most to distort; a fair reading of our results is that they are a strong statement about this task and only a weak one about BN in general.

Why the SVD layer wins under noise and loses on real backgrounds. Both outcomes follow from the same property. The layer keeps the k directions along which the *batch* varies most and deletes the rest. When the signal is a smooth low-dimensional family (a blob with two degrees of freedom) and the corruption is isotropic pixel noise, the signal concentrates in the leading singular directions and the noise spreads across the trailing ones, so truncation is close to an optimal denoiser and the network gains 1.6–3.0 $\times$. When the background is a real MNIST digit, the clutter is structured and occupies leading directions of its own; truncating then removes signal along with noise, and the plain network — free to use all M directions — wins by 5.7 $\times$. The rank constraint is a prior, and it pays exactly when the prior is right. This is also why the same layer produced opposite verdicts in Reports 1–2 and in the final study: a clean, noiseless task has no noise for the projection to remove, so the prior costs without buying.

Placement was the decisive design choice. The reversal between Reports 1–2 and the final result came from moving the layer, not from changing it. As a single bottleneck at the flatten point the layer is an information choke on the entire representation and can only subtract; distributed after every convolution — BN’s own position — it is a mild, repeated denoising step, and the network has capacity between the layers to compensate for what each one removes. The practical lesson generalizes beyond this project: a normalization mechanism should be benchmarked in the slot the incumbent occupies, otherwise the comparison measures the placement rather than the mechanism.

The unfinished half of the derivation. Equation (5) shows the layer decorrelates the retained coordinates exactly but leaves their scales free, so it delivers half of a whitening transform. The symptom shows up in the experiments as sensitivity to how well a given run happens to be conditioned — most visibly in the autoencoder study, where three of eight seeds converged to a poorer optimum and their generative advantage collapsed from 27 $\times$ to 3 $\times$. Equation (6) gives the completed transform in closed form, $\tilde{\mathbf{X}} = \sqrt{N} \mathbf{U}_k \mathbf{V}_k^\top$: keep the directions the SVD chooses, but force the retained singular values to a common scale. That is a genuine *normalization* in the sense of Eq. (1), obtained without a single learnable parameter, and testing it is the natural continuation of this project.

An unexpected finding worth recording. The instability we spent Week 1 worrying about was ours, not the method’s (Section 4.7). A hand-written analytic SVD backward pass looked like the obvious culprit given the $1/(\sigma_i - \sigma_j)$ terms, and the literature supports that suspicion [6]; but a controlled head-to-head over identical seeds showed zero solver failures and zero NaNs on both paths, and the blow-ups vanished once we switched to native autograd. It is a small result, but it is the kind that decides whether a method looks usable.

6 Limitations and Future Work

We state the limitations in order of how much they constrain the conclusions.

- **One task family, one architecture.** Every controlled number in this report comes from Gaussian point sources on a 64×64 grid, regressed by a three-block convolutional network. The problem has exactly two degrees of freedom, which is close to the best case a rank- k layer can be given. *Next step:* run the same three-way comparison on CIFAR-10 classification with a standard ResNet, where BN is at its strongest and the intrinsic dimensionality is unknown — the SVD layer is already packaged as a drop-in module, so this is a matter of compute rather than of new code.
- **The rank was never swept in the final configuration.** The monotone rank curve of Section 4.4 was measured with a single bottleneck, but all headline results use $k = 8$ after *every* convolution, where the effective ranks of successive layers compound. *Next step:* sweep $k \in \{1, 2, 4, 8, 16, 32\}$ at $N_{\text{tr}} \in \{25, 200\}$ under noise, and separately sweep per-layer ranks, to find whether the optimum tracks the intrinsic dimensionality of the data or the batch size.
- **The normalized variant is derived but untested.** Equation (6) is the layer this project was named after and it has not been run. *Next step:* implement $\tilde{\mathbf{X}} = \sqrt{N} \mathbf{U}_k \mathbf{V}_k^\top$ (a two-line change to the forward pass), add the optional learnable γ, β that BN has, and rerun Table 2. This is the highest-value missing experiment in the project.
- **The rank constraint has a training-stability problem.** Three of eight autoencoder seeds converged to a visibly worse optimum, and the quality of the representation degraded with them; we did not investigate why. *Next step, in increasing order of effort:* warm up the network for a few epochs before switching the layer on; average the basis over a short window instead of recomputing it per batch; and check whether the poor runs recover at a lower learning rate.
- **Batch-size dependence was not measured.** The layer inherits BN’s dependence on the batch — and adds a hard cap, since the achievable rank is $\min(M, N)$, so k can never exceed the batch size. All results use $N = 32$ with $k = 8$. *Next step:* sweep the batch size at fixed k , which is where Layer- and GroupNorm-style batch-independent variants of the idea would become interesting.
- **Statistical treatment.** We report mean $\pm$ std over up to 32 seeds and quote per-seed ranges, which is enough to establish the sign of every effect and the order of magnitude of its size, but we do not report paired significance tests. *Next step:* paired tests across seeds and bootstrap confidence intervals on the ratios, so that improvement factors can be quoted with intervals rather than ranges.
- **Cost was not benchmarked.** An SVD per layer per step is more expensive than BN, and although it was not the bottleneck at our scale (Section 3.1), we did not measure wall-clock overhead. *Next step:* report time per epoch against BN as a function of M and N , and test a randomized or truncated SVD as a cheaper substitute.

7 Conclusion

We set out to determine whether the singular value decomposition, on its own, can normalize a batch as well as Batch Normalization, and we built a standalone SVD layer to find out. Analytically, the layer imposes a rank- k degenerate distribution on the batch whose retained coordinates are exactly uncorrelated with second moments σ_i^2/N (Eq. 5) — it decorrelates in a data-adaptive basis where BN merely rescales in a fixed one, but it leaves the scale of the retained directions free,

which is its one structural weakness and points directly at the normalized variant $\tilde{\mathbf{X}} = \sqrt{N}\mathbf{U}_k\mathbf{V}_k^\top$. Empirically, across five report cycles and several hundred controlled runs from bit-identical initial weights, the SVD layer beat Batch Normalization in every comparison we ran — clean, noisy, small-data, real backgrounds, and inside an autoencoder — by $1.5\times$ to $15\times$. Under heavy input noise it beat even the unregularized network at every training-set size we tested, by $1.6\times$ to $3.0\times$ over 32 seeds, with the tightest seed spread of the three models; and in the autoencoder setting a rank-2 layer produced valid random samples $18.0\times$ more often than an unconstrained model, on 8 of 8 seeds. The one honest counterweight is that on real, structured backgrounds the rank constraint removes signal along with noise and the plain network wins. The takeaway we want a reader to keep is that the SVD layer is not a drop-in accuracy booster but a *prior*: it asserts that the batch lives in a low-dimensional subspace, and it pays — handsomely, and more reliably than BN — exactly when that assertion is true.

A The SVD layer in PyTorch

The layer is a plain `nn.Module` with no learnable parameters, so it composes with anything. Listing 1 shows the numerically robust decomposition, and Listing 2 the core of the forward pass; the full implementation, including the inference-basis bank and input validation, is in the repository.

```

1 def stable_svd(X, full_matrices=False):
2     try:
3         if X.is_cuda:
4             return torch.linalg.svd(X, full_matrices=full_matrices, driver='gesvd')
5         return torch.linalg.svd(X, full_matrices=full_matrices)
6     except torch.linalg.LinAlgError:
7         U, S, Vh = torch.linalg.svd(X.cpu(), full_matrices=full_matrices)
8         return U.to(X.device), S.to(X.device), Vh.to(X.device)

```

Listing 1 Robust SVD wrapper. The cuSOLVER default (`gesvdj`) can fail to converge on ill-conditioned or repeated-singular-value matrices, which grid-structured image data produce readily; gradients flow through both paths.

```

1 X = torch.movedim(x, 0, -1).reshape(-1, batch_size) # (M, N), one column per sample
2
3 if self.training:
4     U, S, Vh = self._svd(X)
5     r = min(self.rank, S.shape[0])
6     basis = U[:, :r] # U_k
7     coeffs = S[:r].unsqueeze(1) * Vh[:r] # A = Sigma_k V_k^T
8     X_hat = basis @ coeffs # rank-k reconstruction
9     self._basis_bank.append(basis.detach().cpu()) # bank for the inference basis
10 else:
11     basis = self.inference_basis.to(X.device, X.dtype)
12     coeffs = basis.T @ X # projection onto the fixed basis
13     X_hat = basis @ coeffs
14
15 output = X_hat.reshape(*original_shape[1:], batch_size)
16 output = torch.movedim(output, -1, 0)

```

Listing 2 Core of the forward pass. Training recomputes the basis per batch and banks it; evaluation projects onto the finalized basis, so the layer is deterministic and batch-independent at test time.

```

1 def make_feature_extractor(depth=3, norm=None, rank=8):
2     channels = [1, 8, 16, 32]
3     layers = []
4     for i in range(depth):
5         layers.append(nn.Conv2d(channels[i], channels[i + 1], kernel_size=3, padding=1))
6         if norm == "bn":
7             layers.append(nn.BatchNorm2d(channels[i + 1]))
8         elif norm == "rr":
9             layers.append(RRLayer(rank=rank)) # the SVD layer, same slot as BN
10        layers.append(nn.ReLU(inplace=True))

```

```

11     return nn.Sequential(*layers)
12
13     torch.manual_seed(seed); m_oa = OA(depth=3)           # no normalization
14     torch.manual_seed(seed); m_bn = OA_BN(depth=3)         # BatchNorm2d after every conv
15     torch.manual_seed(seed); m_rr = OA_RR(rank=8, depth=3) # SVD layer after every conv

```

Listing 3 Building the three networks compared in this report. Re-applying the same seed before each constructor guarantees bit-identical convolutional and linear weights, so the slot is the only difference.

References

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer Normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [2] Nils Bjorck, Carla P. Gomes, Bart Selman, and Kilian Q. Weinberger. Understanding Batch Normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [3] Carl Eckart and Gale Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936.
- [4] Lei Huang, Dawei Yang, Bo Lang, and Jia Deng. Decorrelated Batch Normalization. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [5] Sergey Ioffe and Christian Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015.
- [6] Catalin Ionescu, Orestis Vantzos, and Cristian Sminchisescu. Matrix Backpropagation for Deep Networks with Structured Layers. In *IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [7] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- [8] Ping Luo, Xinjiang Wang, Wenqi Shao, and Zhanglin Peng. Towards Understanding Regularization in Batch Normalization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [9] Jad Mounayer, Sebastian Rodriguez, Chady Ghnatios, Charbel Farhat, and Francisco Chinesta. Rank Reduction Autoencoders. *arXiv preprint arXiv:2405.13980*, 2024. <https://arxiv.org/abs/2405.13980>.
- [10] Jad Mounayer. `RR_layer`: reference PyTorch implementation of the Rank Reduction layer. https://github.com/JadM133/RR_layer. Accessed 2026-08-24.
- [11] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [12] Shibani Santurkar, Dimitris Tsipras, Andrew Ilyas, and Aleksander Madry. How Does Batch Normalization Help Optimization? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [13] Yuxin Wu and Kaiming He. Group Normalization. In *European Conference on Computer Vision (ECCV)*, pages 3–19, 2018.